Введение в концепции БД и реляционную модель

Agenda (План занятия)

Зачем нужны базы данных? Эволюция хранения данных.

Что такое СУБД? Их преимущества и виды.

Реляционная модель данных: основы (таблицы, строки, столбцы).

Сущности, атрибуты, связи.

Ключи и целостность данных (Первичный и Внешний ключ).

Про меня

Сертифицированный преподаватель Яндекс Образования по frontend- и backend-разработке (2024 и 2025)

Веду дисциплины:

Базы данных, Анализ больших данных, Разработка IT-проектов

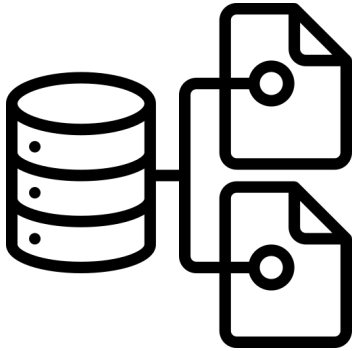
Эволюция данных и преимущества СУБД

Эволюция хранения данных

Данные в файлах (.txt, .csv).

Проблемы: **избыточность**, **несогласованность**, **зависимость** от программ, **сложность одновременного доступа**.

Базы данных и СУБД



Централизованное управление данными.

Системы управления базами данных (СУБД) как решение проблем файловых систем.

Что такое СУБД? (Database Management System)

Системы управления базами данных (СУБД) — это программный комплекс для создания, ведения и совместного использования баз данных многими пользователями.

Создание и поддержка

Централизованное ведение

Совместное использование

Ключевые преимущества СУБД

Отсутствие избыточности данных (или контролируемая избыточность).

Непротиворечивость и целостность данных.

Совместный доступ и управление параллельными операциями.

Безопасность (разграничение прав доступа).

Абстракция данных (пользователь работает с логическим представлением, а не с физическим хранением).

Резервное копирование и восстановление.

Виды СУБД

Иерархические

Сетевые

Реляционные (MySQL, PostgreSQL, Oracle) — фокус нашего курса

NoSQL (MongoDB, Redis) — документные, ключ-значение, графовые.

NewSQL

Реляционная модель данных

Реляционная модель: Основные понятия

База данных (Database)

это не просто файл. Это организованная коллекция структурированных данных, управляемая СУБД. Её можно рассматривать как контейнер, который хранит все связанные между собой объекты.

Схема (Schema)

это логический контейнер внутри базы данных, который группирует связанные между собой объекты (таблицы, представления, процедуры).

Реляционная модель: Основные понятия

Таблица (Table / Relation)

это фундаментальный объект хранения данных. Представляет собой двумерную структуру, состоящую из строк и столбцов.

Столбец (Column / Attribute)

это вертикальная составляющая таблицы. Каждый столбец определяет характеристику сущности и имеет строго определённый тип данных.

Строка (Row / Tuple)

это горизонтальная составляющая таблицы. Каждая строка представляет собой конкретный экземпляр сущности, факт.

Физическое vs. Логическое представление

Логический уровень (то, что видим мы)

- Мы видим базу данных как набор таблиц со строками и столбцами.
- Мы оперируем такими понятиями, как INTEGER или VARCHAR(50).

Физический уровень (то, чем управляет СУБД)

- Данные на диске хранятся в специальных файлах с сложной структурой.
- СУБД разбивает данные на страницы (pages) — блоки фиксированного размера (например, 8 КБ), которые являются единицей ввода/вывода между диском и оперативной памятью.
- Строки таблиц физически размещаются внутри этих страниц. СУБД сама решает, какую строку в какую страницу поместить.
- Для ускорения поиска СУБД создаёт индексы — отдельные структуры данных (чаще всего B-деревья), которые хранят копии части данных (значения ключевых столбцов) и указатели на соответствующие строки в таблицах.

Ключи и ограничения целостности

Первичный ключ (Primary Key, PK)

Первичный ключ (Primary Key, PK) — это не просто «столбец с ID». Это минимальный набор атрибутов (столбцов), который **однозначно идентифицирует каждую строку в таблице (обеспечивает уникальность и идентифицируемость каждой записи в таблице)**.

Суррогатный ключ
(Surrogate Key)

Естественный ключ
(Natural Key)

Внешний ключ (Foreign Key, FK)

Внешний ключ (Foreign Key, FK) — это не самостоятельный объект. Это **ограничение** (constraint), которое накладывается на столбец или группу столбцов в одной таблице и которое требует, чтобы каждое значение в этом столбце совпадало со значением первичного ключа (или уникального ключа) в другой таблице.

Ограничения целостности (Constraints)

Ограничение целостности — это правило, предписанное схеме базы данных, которое ограничивает множество допустимых состояний БД. Проще говоря, это правила, которые данные не могут нарушить.

Целостность по сущностям

Целостность по сущностям (Entity Integrity) гарантирует, что каждая сущность (строка в таблице) будет уникальна и идентифицируема.

Обеспечивается через ограничение PRIMARY KEY (реже — UNIQUE NOT NULL).

Ссылочная целостность

Ссылочная целостность (Referential Integrity) гарантирует, что связи между таблицами не разорвутся, и в базе данных не появятся «висячие» ссылки.

Обеспечивается через ограничение FOREIGN KEY.

Целостность по атрибутам

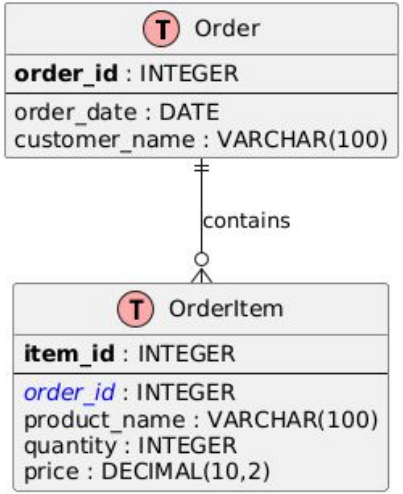
Целостность по атрибутам (Domain Integrity) гарантирует, что значение в каждом отдельном столбце является допустимым и осмысленным с точки зрения его домена (множества допустимых значений).

Обеспечивается через несколько механизмов:

- **Тип данных** (Data Type) — первая и главная линия обороны. Столбец, объявленный как INTEGER, не примет текст "abc". Тип данных ограничивает вид хранимой информации.
- Ограничение **CHECK** позволяет задать произвольное логическое условие, которое должно быть истинно для значения в столбце (или для всей строки)."
- Ограничение **NOT NULL** запрещает отсутствие значения, делая атрибут обязательным для заполнения.
- Ограничение **UNIQUE** гарантирует, что все значения в столбце (или комбинации столбцов) различны.

Типы связей между отношениями (сущностями)

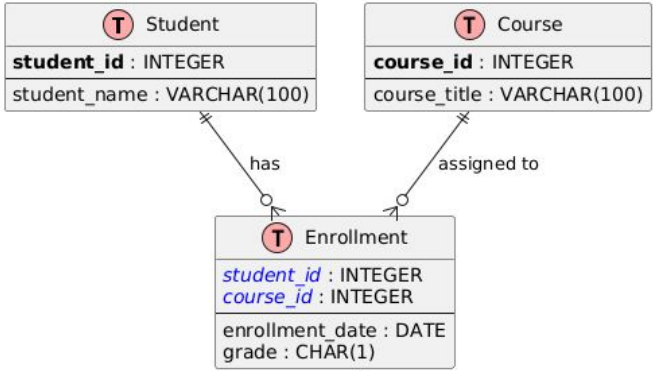
Связь «один ко многим» (One-to-Many, 1:N)



Одна запись в таблице Order может быть связана с несколькими записями в таблице OrderItem, но каждая запись в таблице OrderItem связана только с одной записью в таблице Order.

В таблице на стороне «многих» (таблица OrderItem) создается внешний ключ (Foreign Key), который ссылается на первичный ключ (Primary Key) таблицы на стороне «один» (таблица Order).

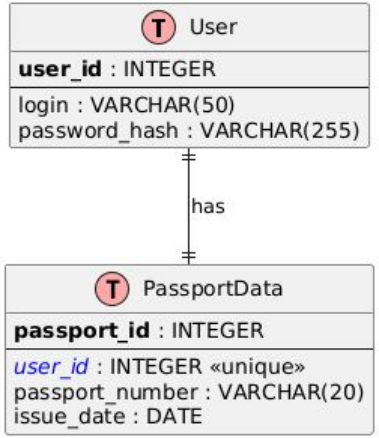
Связь многие ко многим» (Many-to-Many, M:N)



Одна запись в таблице *Students* может быть связана с несколькими записями в таблице *Course*, и наоборот, одна запись в таблице *Course* может быть связана с несколькими записями в таблице *Students*.

Этот тип связи **не может быть реализован напрямую двумя таблицами**. Требуется создание промежуточной таблицы-связки (Junction Table, Associative Entity) *Enrollment*. Эта таблица обычно содержит как минимум два внешних ключа, ссылающихся на первичные ключи таблиц Students и Course. Её собственный первичный ключ часто является композитным (составным) из этих двух внешних ключей.

Связь «один к одному» (One-to-One, 1:1)



Одна запись в таблице User связана только с одной записью в таблице PassportData, и наоборот.

В одной из таблиц создается внешний ключ, который ссылается на первичный ключ другой таблицы. **При этом для этого внешнего ключа добавляется ограничение уникальности (UNIQUE)**, чтобы гарантировать, что связь будет только одна.